



HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch



Laboratoire 7 : MSS Simple - Détection clic/double clic

Département : Technologies de l'Information et de la Communication (TIC)
Unité d'enseignement : Conception de systèmes numériques (CSN)

Auteurs : Fabien Léger & Jonathan Nicolet
Professeur : E. Messerli
Assistant : B. Sutterlet
Classe : CSN-A
Date : 27 mai 2026

1 Introduction

Le but de ce laboratoire est d'implémenter un système de détection de clic et de double clic. Ce système sera réalisé sous la forme d'une machine à états séquentielle synchrone (MSS). Il faudra la décomposer en une Unité de Traitement (UT) et une Unité de Contrôle (UC).

L'unité de traitement servira comme compteur pour décompter le temps et changer ses sorties en fonction des entrées du système.

L'unité de contrôle permettra de lancer, contrôler et vérifier les sorties de l'unité de traitement.

Une première phase consistera à analyser le système afin de comprendre son fonctionnement grâce à des chronogrammes. Cela permettra d'établir un schéma bloc et d'en indiquer les contraintes des entrées et des sorties.

Dans une deuxième phase, il faudra analyser, concevoir et réaliser le timer. On sera ainsi capable de le simuler et de vérifier son fonctionnement.

Une troisième phase nous amènera à créer l'unité de contrôle en faisant son graphe des états et sa description VHDL. Cela sera complété par une synthèse et une vérification du schéma généré.

En dernière phase, nous aurons une mise en commun du système avec une simulation automatique, suivie d'une intégration sur carte pour vérifier le fonctionnement réel de notre programme.

2 Analyse et conception

Voici tout d'abord les entrées et sorties du système à réaliser.

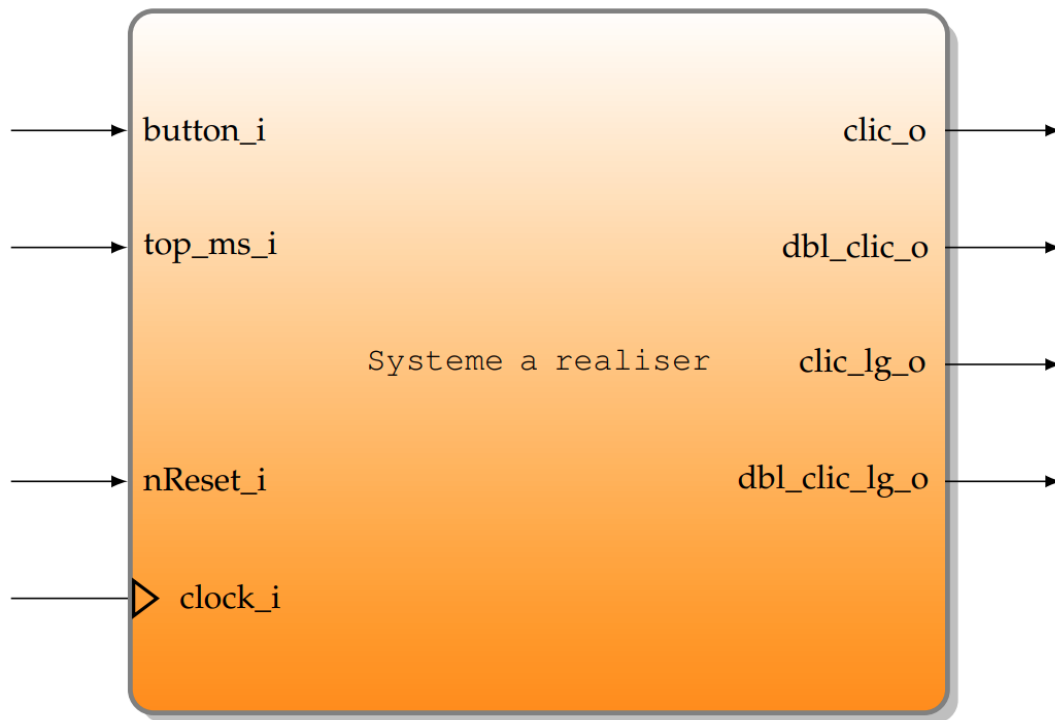


FIGURE 1 – Bloc du système à réaliser avec ses entrées et sorties

TABLE 1 – Description des signaux du module

Nom	Dir.	Taille	Description
<code>nReset_i</code>	in	1	Remise à zéro asynchrone du système, active à l'état bas
<code>clock_i</code>	in	1	Horloge du système, 1 MHz
<code>button_i</code>	in	1	Indique si le bouton est enfoncé ou non, actif à l'état haut
<code>top_ms_i</code>	in	1	Impulsion d'une période d'horloge générée chaque milliseconde, à une fréquence de 1 kHz
<code>clic_o</code>	out	1	Indique la détection d'un clic, actif durant un coup d'horloge
<code>dbl_clic_o</code>	out	1	Indique la détection d'un double-clic, actif durant un coup d'horloge
<code>clic_lg_o</code>	out	1	Indique la détection d'un clic, actif durant une demi-seconde
<code>dbl_clic_lg_o</code>	out	1	Indique la détection d'un double-clic, actif durant une demi-seconde

Le fonctionnement attendu du système est le suivant :

- un clic est valide si le bouton `button_i` a été pressé pendant une durée inférieure à $T1 = 0.3 \text{ s}$;
- un double-clic est détecté si :
 - la première condition a été remplie ;
 - le bouton a été relâché pendant un temps ne dépassant pas $T2 = 0.2 \text{ s}$;
 - le bouton a ensuite été enfoncé à nouveau en respectant la première condition.
- un clic simple ne peut être validé que si aucun double-clic n'a été détecté.

On comprend donc qu'après l'action d'un premier clic, ce clic ne peut pas durer plus de $T1$. Si cette limite est dépassée, le système retourne dans un état d'attente.

Dans le cas où le bouton est relâché avant cette limite, le système attend un éventuel deuxième clic pendant une durée maximale de $T2$. Si aucun deuxième clic n'est détecté durant cet intervalle, un clic simple est généré.

Finalement, si le premier clic a duré moins de $T1$, que le bouton est resté relâché au maximum pendant $T2$, et que le deuxième clic respecte également la limite de $T1$, alors un double-clic est détecté. Dans le cas contraire, si le double-clic n'est pas validé, alors la séquence est invalide et aucun clic n'est enregistré.

2.1 Contraintes du système

Le système est entièrement synchrone avec une horloge principale de 1 MHz. Le signal `top_ms_i`, de fréquence 1 kHz, sert de référence temporelle et est actif durant un seul cycle d'horloge.

Le signal `button_i` est asynchrone par rapport à l'horloge système et doit être synchronisé avant son utilisation dans la machine d'état afin d'éviter tout problème de métastabilité.

Le reset (`nReset_i`) est asynchrone et actif à l'état bas.

Les signaux `start_o`, `clic_o` et `dbl_clic_o` doivent être générés sous forme d'impulsions d'un seul cycle d'horloge afin de garantir un comportement correct et synthétisable.

Les sorties `clic_lg_o` et `dbl_clic_lg_o` sont simplement les mêmes sorties pour le clic et le double clic ci-dessus, mais maintenues pendant une demi-seconde.

La machine d'état doit garantir qu'un clic simple n'est validé que s'il n'y a pas de double-clic détecté. De plus, la détection du double-clic doit respecter une contrainte temporelle stricte définie par le timer ($T1$ et $T2$).

Enfin, le système doit rester entièrement synchrone afin de permettre une synthèse correcte et l'évaluation de la fréquence maximale (F_{max}).

2.2 Chronogrammes

Nous voulons donc d'abord détecter un premier clic valide, c'est-à-dire un clic dont la durée est inférieure à $T1$. Les ... dans les chronogrammes indiquent que du temps passe sans qu'un événement important pour l'analyse ne se produise. `clic_valide` est utilisé pour montrer ce qu'il se passe dans le système. Il n'existera pas dans la version finale. Il peut cependant être vu comme une transition d'état dans la MSS.

Dans ces exemples, nous avons repris selon la donnée les informations suivantes.

- $T1 = 300\text{ms}$
- $T2 = 200\text{ms}$

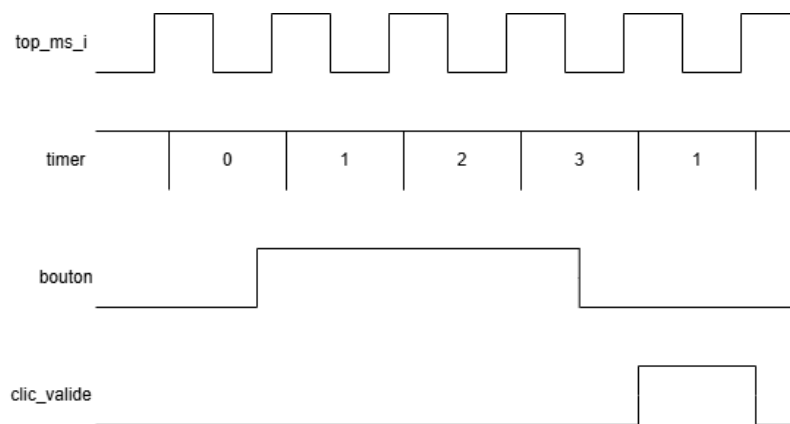


FIGURE 2 – Premier clic valide

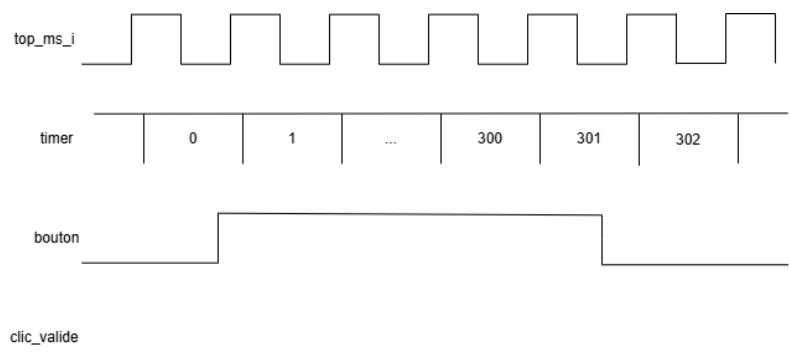


FIGURE 3 – Premier clic non valide

Dans le cas où un premier clic valide est détecté, le système attend un éventuel deuxième clic pendant un temps $T2$. Si aucun deuxième clic n'arrive durant ce délai, un clic simple est généré.

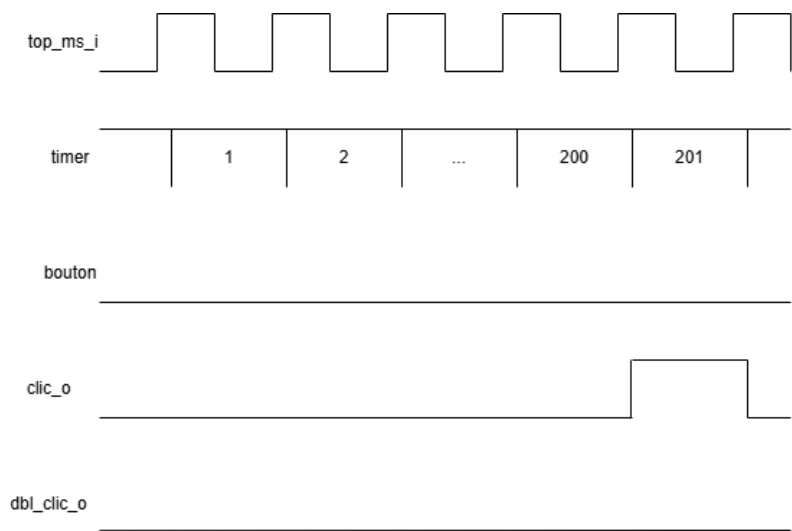


FIGURE 4 – Validation d'un clic simple

Cependant, si un nouveau clic arrive à temps, le système vérifie s'il est valide. Si c'est le cas, un double-clic est généré. Sinon, aucun clic n'est généré.

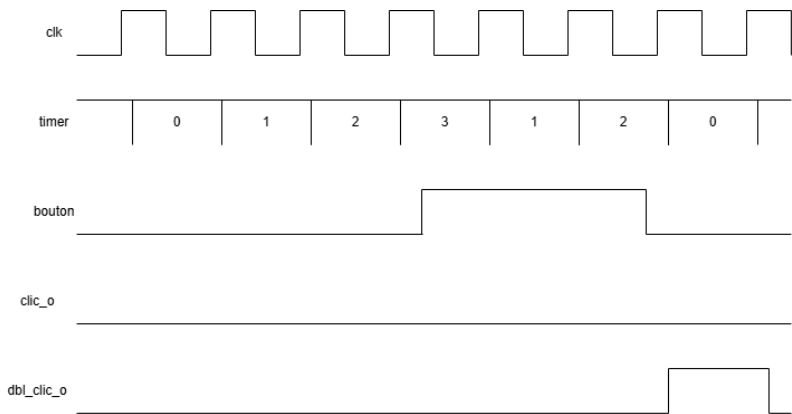


FIGURE 5 – Double-clic valide

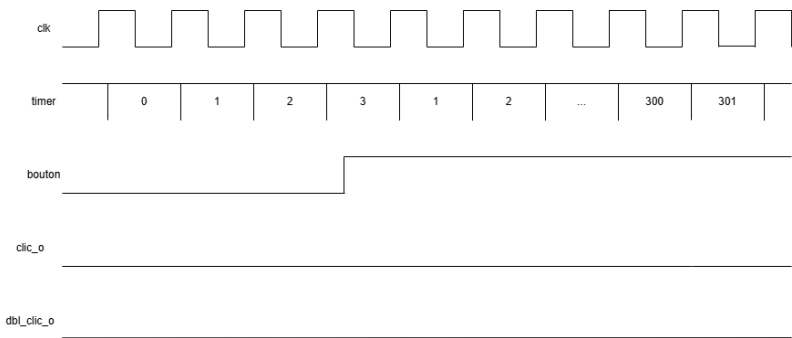


FIGURE 6 – Double-clic non valide, pas de clic généré

2.3 Schéma

Le système est composé d'un bloc timer, d'une machine d'états, de deux blocs de maintien et d'une flip-flop D afin de synchroniser le signal du bouton.

Le timer envoie les signaux `trigger1_o` et `trigger2_o` à la machine d'états lorsque les limites temporelles sont atteintes. Le fonctionnement détaillé du timer est présenté dans la section 3.

La machine d'états 4 reçoit l'état du bouton ainsi que les signaux de déclenchement du timer. Elle détermine alors s'il faut relancer le compteur, attendre un nouvel événement ou activer l'une des deux sorties de détection, à savoir `clic_o` ou `dbl_clic_o`.

Finalement, les blocs de maintien permettent de prolonger la durée d'activation des signaux de clic et de double-clic pendant `0.5 s`, afin de les rendre visibles sur les sorties longues.

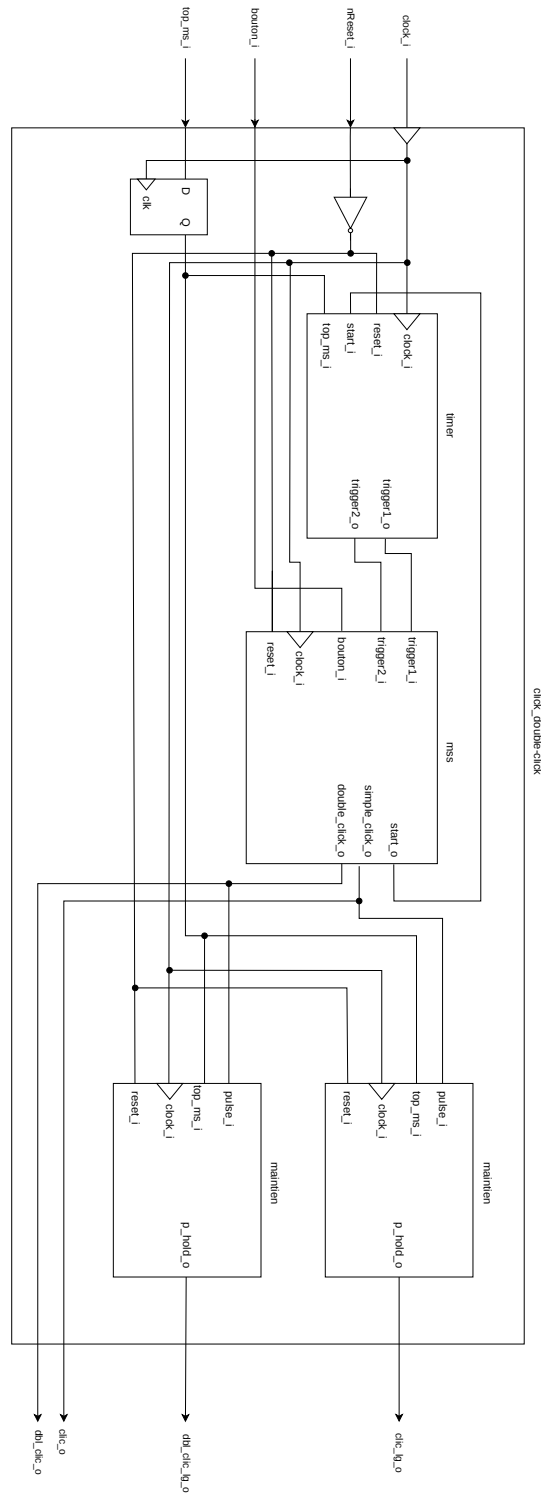


FIGURE 7 – Schéma bloc du système de détection de clic et double-clic

3 Timer

3.1 Analyse

Lorsque l'entrée `start_i` passe à 1, le compteur est réinitialisé à 0. Ensuite, lorsque `start_i` repasse à 0, le compteur commence à compter à chaque impulsion de `top_ms_i`, jusqu'à atteindre la valeur maximale de 1023, car le compteur est codé sur 10 bits. Une fois cette valeur atteinte, il maintient sa valeur afin de ne pas reboucler automatiquement.

Les seules sorties de notre timer sont :

- `trigger1_o`, activée après un délai de `T1_g` ms ;
- `trigger2_o`, activée après un délai de `T2_g` ms.

Ces signaux sont donc activés lorsque les délais correspondants sont dépassés. Ces constantes sont génériques et peuvent donc être modifiées selon les besoins de la simulation ou de l'implémentation.

Les fonctions synchrones du timer sont les suivantes :

- maintien lorsque `top_ms_i` est à 0 ou lorsque le compteur a atteint sa valeur maximale ;
- incrémentation lorsque `top_ms_i` est à 1 et que le compteur n'a pas encore atteint sa valeur maximale ;
- chargement de 0 lorsque `start_i` est à 1, afin de recommencer le comptage.

<code>start_i</code>	<code>top_ms_i</code>	<code>Q_pres</code>	<code>Q_fut</code>	Commentaire
1	-	-	0	Chargement à 0
0	1	1023	<code>Q_pres</code>	Maintien à la valeur maximale
0	1	-	<code>Q_pres + 1</code>	Incrémentation
0	0	-	<code>Q_pres</code>	Maintien

TABLE 2 – Table des fonctions synchrones du compteur

Lorsqu'on recommence à compter, on charge la valeur 0 afin d'attendre de 0 à `T - 1`, ce qui correspond à une durée de `T`. Une action est ensuite effectuée lorsque la limite temporelle est dépassée.

3.2 Implémentation

Le VHDL du timer est relativement simple. On détermine l'état futur du registre comme indiqué dans la table des fonctions synchrones. On déclare l'élément mémoire, puis on met les sorties à 1 lorsque les limites temporelles sont dépassées.

Listing 1 – Implémentation VHDL du timer

```

1 architecture comport of timer is
2   signal reg_pres_s : unsigned(9 downto 0);
3   signal reg_fut_s  : unsigned(9 downto 0);
4 begin
5
6   -- Future state decoder
7   reg_fut_s <= (others => '0') when start_i = '1' else --
      load 0
8
9       reg_pres_s when reg_pres_s >= 1023 else
10          -- hold (max)
11          reg_pres_s + 1 when top_ms_i = '1' else
12             -- increment
13             reg_pres_s;
14
15                                     -- hold
16
17   -- Description of synchronous register
18   Mem: process (clock_i, reset_i)
19   begin
20     if (reset_i = '1') then
21       reg_pres_s <= (others => '0');
22     elsif rising_edge(clock_i) then
23       reg_pres_s <= reg_fut_s;
24     end if;
25   end process;
26
27   -- Determine the outputs
28   trigger1_o <= '1' when reg_pres_s > T1_g else
      '0';
29   trigger2_o <= '1' when reg_pres_s > T2_g else
      '0';
30 end comport;

```

Une chose à noter est que nous testons si le compteur est strictement plus grand que T1 et T2. En effet, la comparaison avec T2 fait sens selon le cahier des charges mais pas T1 qui devrait être plus grand ou égal.

Lors de la simulation, l'utilisation d'une comparaison supérieure ou égale activait `trigger1_o` un cycle trop tôt par rapport aux transitions attendues par la MSS. L'utilisation d'une comparaison strictement supérieure permet de retarder l'activation du signal d'un cycle et de respecter le comportement observé dans la simulation. Il est possible que cette correction décale certains événements d'un cycle d'horloge.

3.3 Simulation

Les logs de compilation avec QuestaSim sont présentés ci-dessous.

```
# vsim work.console_sim
# Start time: 14:56:59 on May 26,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.det_clc_dbclc_pkg
# Loading ieee.numeric_std(body)
# Loading work.console_sim(struct)#1
# Loading work.timer(comport)#1
# 8.6.5
# 4.0.2
# REDS Console
# REDS_console
# /opt/tools_reds
# linux
# FALSE
# ./REDS_console_sigImg.gif
# 16
# 16
# ** Note: (vsim-8009) Loading existing optimized design _opt
```

FIGURE 8 – Logs de compilation du timer

Le chronogramme de la simulation manuelle est présenté ci-dessous.

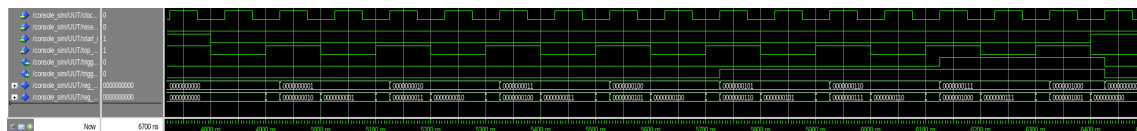


FIGURE 9 – Chronogramme de simulation du timer

Comme on peut le voir sur ce chronogramme, lorsque `start_i` est à 1, le timer est bien remis à 0. Ensuite, lorsque `start_i` repasse à 0, le compteur s'incrémente normalement à chaque impulsion de `top_ms_i`. On observe également que les sorties `trigger1_o` et `trigger2_o` s'activent correctement lorsque les délais correspondants sont dépassés. Elles restent actives jusqu'à ce que le timer soit réinitialisé par une nouvelle activation de `start_i`. Le composant semble donc fonctionner correctement.

4 Machine séquentielle simple

4.1 Graphe des états

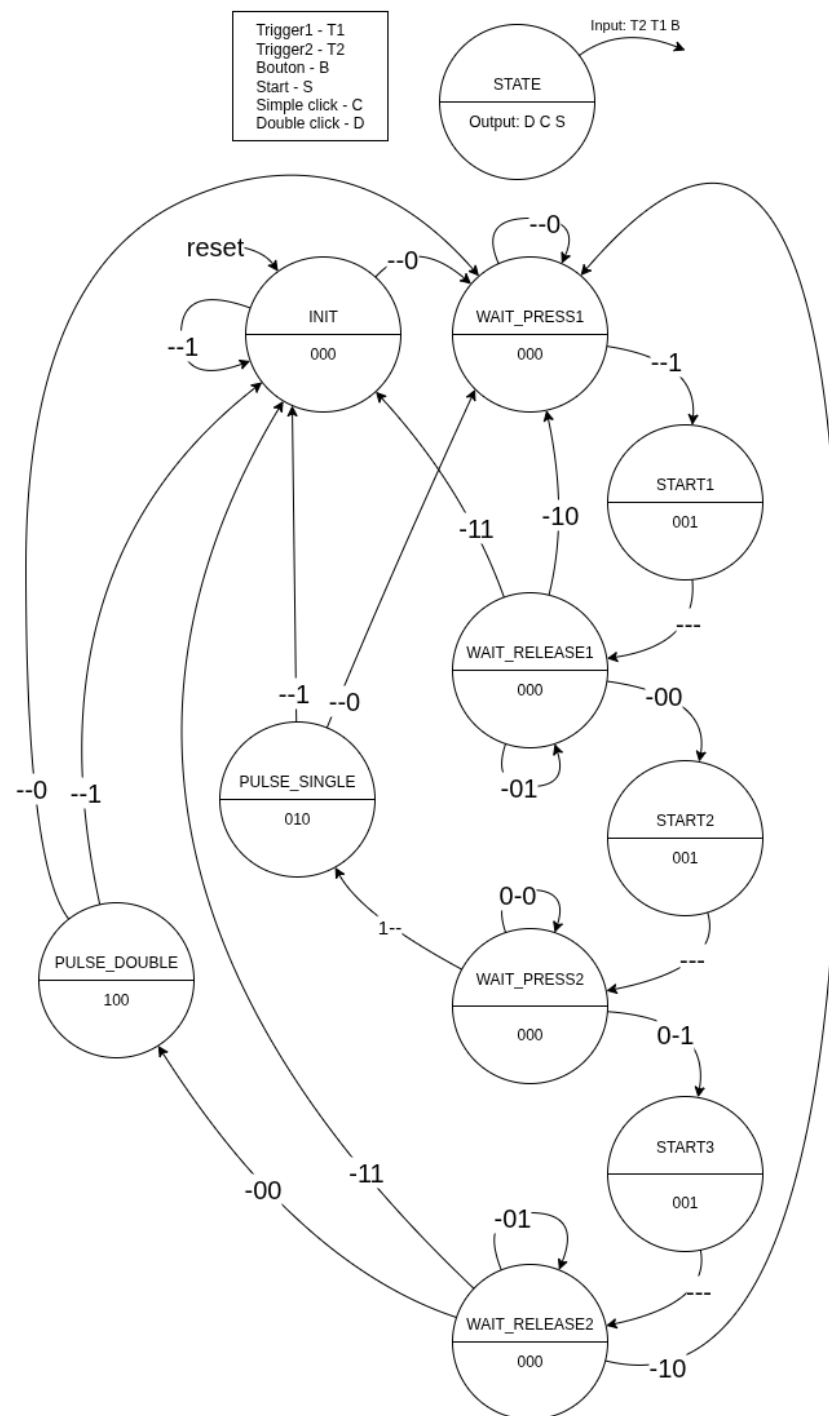


FIGURE 10 – Graphe des états de la MSS

Il a été décidé dans ce graphe d'avoir un état `INIT` au cas où on commence en pressant le bouton. Cela nous permet aussi lorsqu'une séquence est terminée mais que le bouton est déjà/toujours pressé en revenant sur `INIT` afin d'attendre le relâchement. Au final, cet état se révèle nécessaire, sinon il pourrait y avoir un risque de repartir dans une nouvelle séquence en maintenant le bouton enfoncé.

`WAIT_PRESS1` est une fois que le bouton est relâché, en attente d'une nouvelle action pour commencer une séquence.

Les états `START1`, `START2` et `START3` permettent de lancer respectivement le timer de relâchement du premier clic, le timer entre les deux clics et finalement le timer de relâchement du deuxième clic. Il est obligatoire d'avoir ces états transitoires pour n'avoir la sortie `Start` à '1' uniquement durant un cycle d'horloge.

Les états `WAIT_RELEASE1`, `WAIT_PRESS2`, et `WAIT_RELEASE2` représentent les états d'attente suivant les différents états de démarrage.

Finalement, `PULSE_SINGLE` et `PULSE_DOUBLE` permettent d'envoyer un signal de détection de clic simple ou double.

On obtient ainsi un total de 10 états.

4.2 Implémentation VHDL

La machine d'états est décrite en VHDL à l'aide de deux processus distincts. Le premier processus est combinatoire et permet de déterminer l'état futur ainsi que les sorties de la MSS. Le second processus est séquentiel et mémorise l'état courant sur front montant de l'horloge.

4.2.1 Déclaration des états

Les états de la machine sont encodés sur 4 bits. Cet encodage permet de représenter les 10 états nécessaires au fonctionnement de la MSS.

Listing 2 – Déclaration des états de la MSS

```
1 signal state_pres_s : std_logic_vector(3 downto 0);
2 signal state_fut_s  : std_logic_vector(3 downto 0);
3
4 constant INIT          : std_logic_vector(3 downto 0) :=
   "0000";
5 constant WAIT_PRESS1   : std_logic_vector(3 downto 0) :=
   "0001";
6 constant START1        : std_logic_vector(3 downto 0) :=
   "0010";
7 constant WAIT_RELEASE1 : std_logic_vector(3 downto 0) :=
   "0011";
8 constant START2        : std_logic_vector(3 downto 0) :=
   "0100";
9 constant WAIT_PRESS2   : std_logic_vector(3 downto 0) :=
   "0101";
10 constant START3       : std_logic_vector(3 downto 0) :=
   "0110";
11 constant WAIT_RELEASE2 : std_logic_vector(3 downto 0) :=
   "0111";
12 constant PULSE_SINGLE  : std_logic_vector(3 downto 0) :=
   "1000";
13 constant PULSE_DOUBLE  : std_logic_vector(3 downto 0) :=
   "1001";
```

Les états non utilisés sont traités dans le cas `others`, ce qui permet de revenir dans un état connu si la MSS se retrouve dans une combinaison invalide.

4.2.2 Processus combinatoire

Le processus combinatoire détermine l'état futur de la machine en fonction de l'état présent, de l'état du bouton et des signaux `trigger1_i` et `trigger2_i` générés par le timer.

Listing 3 – Structure du processus combinatoire

```
1 Fut: process (trigger1_i, trigger2_i, bouton_i, state_pres_s)
2 begin
3
4     state_fut_s <= INIT;
5
6     start_o <= '0';
7     simple_click_o <= '0';
8     double_click_o <= '0';
9
10    case state_pres_s is
11
12        -- Etats de la MSS
13
14        when others =>
15            null;
16
17    end case;
18 end process;
```

Les sorties sont mises à 0 par défaut afin d'éviter la création de latches. Elles ne sont activées que dans les états où cela est nécessaire. Par exemple, `start_o` est uniquement activé dans les états de démarrage du timer, tandis que `simple_click_o` et `double_click_o` sont uniquement activés dans les états d'impulsion.

4.2.3 État initial et attente du premier appui

L'état INIT permet d'attendre que le bouton soit relâché. Cet état est utile au démarrage du système, mais également lorsqu'une séquence invalide se termine alors que le bouton est encore pressé.

Listing 4 – États INIT et WAIT_PRESS1

```
1 when INIT =>
2   if (bouton_i = '0') then
3     state_fut_s <= WAIT_PRESS1;
4   else
5     state_fut_s <= INIT;
6   end if;
7
8 when WAIT_PRESS1 =>
9   if (bouton_i = '1') then
10    state_fut_s <= START1;
11  else
12    state_fut_s <= WAIT_PRESS1;
13  end if;
```

Lorsque le bouton est relâché, la MSS passe dans l'état WAIT_PRESS1. Cet état attend le premier appui du bouton. Dès qu'un appui est détecté, la machine passe dans l'état START1 afin de démarrer le timer.

4.2.4 Démarrage du timer

Les états START1, START2 et START3 servent uniquement à activer la sortie start_o pendant un cycle d'horloge.

Listing 5 – États de démarrage du timer

```
1 when START1 =>
2   start_o <= '1';
3   state_fut_s <= WAIT_RELEASE1;
4
5 when START2 =>
6   start_o <= '1';
7   state_fut_s <= WAIT_PRESS2;
8
9 when START3 =>
10  start_o <= '1';
11  state_fut_s <= WAIT_RELEASE2;
```


4.2.5 Validation du premier clic

Après le premier appui, la MSS attend le relâchement du bouton dans l'état `WAIT_RELEASE1`. Si le bouton est relâché avant le dépassement de `T1`, le premier clic est considéré comme valide.

Listing 6 – Validation du premier clic

```
1 when WAIT_RELEASE1 =>
2   if (trigger1_i = '0' and bouton_i = '0') then
3     state_fut_s <= START2;
4   elsif (trigger1_i = '0' and bouton_i = '1') then
5     state_fut_s <= WAIT_RELEASE1;
6   elsif (trigger1_i = '1' and bouton_i = '0') then
7     state_fut_s <= WAIT_PRESS1;
8   else
9     state_fut_s <= INIT;
10  end if;
```

Si `trigger1_i` est actif, cela signifie que la durée maximale du clic a été dépassée. Dans ce cas, la séquence est invalidée. Si le bouton est déjà relâché, la machine retourne directement dans l'état `WAIT_PRESS1`. Si le bouton est encore pressé, elle retourne dans `INIT` afin d'attendre le relâchement.

4.2.6 Attente d'un deuxième clic

Lorsque le premier clic est valide, le timer est relancé avec l'état `START2`. La MSS passe ensuite dans l'état `WAIT_PRESS2`, qui attend un éventuel deuxième appui.

Listing 7 – Attente du deuxième appui

```
1 when WAIT_PRESS2 =>
2   if trigger2_i = '1' then
3     state_fut_s <= PULSE_SINGLE;
4   elsif bouton_i = '1' then
5     state_fut_s <= START3;
6   else
7     state_fut_s <= WAIT_PRESS2;
8   end if;
```

Si aucun deuxième appui n'est détecté avant le dépassement de `T2`, la machine valide un clic simple. Si un deuxième appui est détecté à temps, la machine passe dans l'état `START3` afin de relancer le timer pour mesurer la durée du deuxième clic.

4.2.7 Validation du double-clic

L'état WAIT_RELEASE2 permet de vérifier si le deuxième clic respecte également la durée maximale T1.

Listing 8 – Validation du deuxième clic

```
1 when WAIT_RELEASE2 =>
2   if (trigger1_i = '1' and bouton_i = '1') then
3     state_fut_s <= INIT;
4   elsif (trigger1_i = '1' and bouton_i = '0') then
5     state_fut_s <= WAIT_PRESS1;
6   elsif bouton_i = '0' then
7     state_fut_s <= PULSE_DOUBLE;
8   else
9     state_fut_s <= WAIT_RELEASE2;
10  end if;
```

Si le bouton est relâché avant le dépassement de T1, le double-clic est validé et la machine passe dans l'état PULSE_DOUBLE. Si trigger1_i est actif, le deuxième clic est trop long et la séquence de double-clic est invalidée.

4.2.8 Génération des impulsions de sortie

Les états PULSE_SINGLE et PULSE_DOUBLE permettent de générer respectivement les sorties simple_click_o et double_click_o. Ces sorties ne sont actives que pendant un coup d'horloge.

Listing 9 – Génération des impulsions de clic

```
1 when PULSE_SINGLE =>
2   simple_click_o <= '1';
3   if (bouton_i = '1') then
4     state_fut_s <= INIT;
5   else
6     state_fut_s <= WAIT_PRESS1;
7   end if;
8
9 when PULSE_DOUBLE =>
10  double_click_o <= '1';
11  if (bouton_i = '1') then
12    state_fut_s <= INIT;
13  else
14    state_fut_s <= WAIT_PRESS1;
15  end if;
```

Après l'activation d'une sortie, la MSS retourne soit dans l'état WAIT_PRESS1 si le bouton est relâché, soit dans l'état INIT si le bouton est encore pressé. Cela évite de démarrer une nouvelle séquence tant que le bouton n'a pas été relâché.

4.2.9 Processus de mémorisation

Le deuxième processus est séquentiel. Il mémorise l'état courant de la MSS à chaque front montant de l'horloge. Le reset interne `reset_i` est asynchrone et actif à l'état haut. Il provient de l'inversion du signal externe `nReset_i`, actif à l'état bas.

Listing 10 – Processus de mémorisation de la MSS

```
1 Mem: process (clock_i, reset_i)
2 begin
3     if (reset_i = '1') then
4         state_pres_s <= INIT;
5     elsif rising_edge(clock_i) then
6         state_pres_s <= state_fut_s;
7     end if;
8 end process;
```

Ainsi, la MSS revient dans l'état `INIT` lors d'une remise à zéro. Sinon, à chaque front montant de l'horloge, l'état futur calculé par le processus combinatoire devient le nouvel état présent.

5 Détection de clic / double clic top

Le top contient les différents composants de ce projet. Pour les relier entre eux, on aura besoin de différents signaux internes.

Listing 11 – signaux internes

```
1
2 architecture struct of det_clic_dblclic_top is
3
4     -- Internal signal declarations
5     signal reset_s      : std_logic;
6     signal start_s      : std_logic;
7     signal button_s     : std_logic;
8     signal trigger1_s   : std_logic;
9     signal trigger2_s   : std_logic;
10    signal simple_click_s : std_logic;
11    signal double_click_s : std_logic;
```

Il faut ensuite déclarer les différents composants que l'on va utiliser en se référant à la déclaration vhdl de ceux-ci.

Listing 12 – déclaration des composants

```
1     -- Component declarations
2     -- Treatment Unit
3     component timer
4         generic (
5             T1_g : natural range 1 to 1023 := 300;
6             T2_g : natural range 1 to 1023 := 200
7         );
8     port (
9         clock_i      : in  std_logic;
10        reset_i      : in  std_logic;
11        start_i      : in  std_logic;
12        top_ms_i     : in  std_logic;
13        trigger1_o    : out std_logic;
14        trigger2_o    : out std_logic
15    );
16 end component;
17 for all : timer use entity work.timer;
18
19 -- Control Unit
20 component mss_clic_dblclic
21     port (
22         clock_i      : in  std_logic;
23         reset_i      : in  std_logic;
24         trigger1_i    : in  std_logic;
25         trigger2_i    : in  std_logic;
26         bouton_i     : in  std_logic;
```

```

27         start_o          : out std_logic;
28         simple_click_o   : out std_logic;
29         double_click_o   : out std_logic
30     );
31 end component;
32 for all : mss_clic_dblclic use entity work.mss_clic_dblclic
33     ;
34
35 -- Holding Component
36 component maintien
37     generic (T_HOLD : natural range 1 to 1023 := 2
38         );
39     port (clock_i       : in  std_logic;
40         reset_i        : in  std_logic;
41         pulse_i        : in  std_logic;
42         top_ms_i       : in  std_logic;
43         p_hold_o       : out std_logic
44     );
45 end component;
46 for all : maintien use entity work.maintien;

```

Nous souhaitons ensuite traiter les signaux d'entrée :

- On reçoit un nReset mais les composants utilisent un reset, il faut donc l'inverser.
- Le signal du bouton est asynchrone, il faut donc le synchroniser. On utilise une flip-flop D pour ce faire.

Listing 13 – traitement signaux d'entrée

```

1 begin
2
3     -- Adapt polarity
4     reset_s <= not nReset_i;
5
6     -- Make button_i synchronous through a flip-flop D
7     process(reset_s, clock_i)
8     begin
9         if reset_s = '1' then
10             button_s <= '0';
11         elsif rising_edge(clock_i) then
12             button_s <= button_i;
13         end if;
14     end process;

```

Il faut ensuite instancier tous les composants et leur donner les bons signaux d'entrée et de sortie afin qu'ils communiquent correctement ensemble. Voir le schéma pour les liaisons entre les blocs. On a donc besoin d'un timer, d'une machine d'état et de deux blocs de maintien : un pour chaque type de clic.

Listing 14 – instantiation des composants

```

1  -- Treatment unit to count time after clicks and releases
2  U_timer : timer
3      generic map (T1_g => T1_g,
4                   T2_g => T2_g
5                   )
6      port map (clock_i => clock_i,
7                reset_i => reset_s,
8                start_i => start_s,
9                top_ms_i => top_ms_i,
10               trigger1_o => trigger1_s,
11               trigger2_o => trigger2_s
12               );
13
14  -- Control unit to start the timer and handle outputs
15  mss : mss_clic_dbliclic
16      port map(clock_i      => clock_i,
17              reset_i      => reset_s,
18              trigger1_i    => trigger1_s,
19              trigger2_i    => trigger2_s,
20              bouton_i      => button_s,
21              start_o       => start_s,
22              simple_click_o => simple_click_s,
23              double_click_o => double_click_s
24              );
25
26  -- Holds the single click
27  clic_lg : maintien
28      generic map (T_HOLD => T_HOLD
29                  )
30      port map (clock_i      => clock_i,
31              reset_i      => reset_s,
32              pulse_i       => simple_click_s,
33              top_ms_i      => top_ms_i,
34              p_hold_o      => clic_lg_o
35              );
36
37  -- Holds the double click
38  dbl_clic_lg : maintien
39      generic map (T_HOLD => T_HOLD
40                  )
41      port map (clock_i      => clock_i,
42              reset_i      => reset_s,
43              pulse_i       => double_click_s,
44              top_ms_i      => top_ms_i,
45              p_hold_o      => dbl_clic_lg_o
46              );

```

Finalement, il faut déterminer les signaux de sorties. Ceux-ci prennent simplement leurs signaux internes correspondants. On ne les assigne pas directement depuis les composants, contrairement aux variantes maintenues, car ces signaux internes doivent aussi être réutilisés pour générer les sorties longues. Par convention, il est préférable d'éviter de relire directement un signal de sortie.

Listing 15 – assignation de la sortie

```
1  -- Single clock time outputs
2  clic_o <= simple_click_s;
3  dbl_click_o <= double_click_s;
4
5  end struct;
```

6 Synthèse

On peut maintenant effectuer la synthèse Quartus afin de vérifier que notre solution est correcte et d'estimer son coût matériel.

6.1 Résumé de la synthèse

Entité top	LEs	Pins	Fmax
det_clic_db clic_top.vhd	64	8	52.76 MHz
timer.vhd	21	6	59.56 MHz
mss_clic_db clic.vhd	19	8	89.17 MHz

TABLE 3 – Résumé de la synthèse selon l'entité top utilisée

Le nombre d'éléments logiques semble acceptable pour `det_clic_db clic_top.vhd`, surtout sachant qu'il contient une unité de contrôle, une unité de traitement ainsi que la logique de maintien.

En regardant plus précisément les résultats, le timer et la MSS utilisent un nombre assez proche de LEs. Par contre, le timer a un impact plus important sur la Fmax que la MSS.

Il faut tout de même faire attention au fait que ces résultats ont été obtenus individuellement. Pour l'entité top, il faut encore ajouter les connexions entre les composants ainsi que la logique supplémentaire nécessaire à leur assemblage.

6.2 Vues RTL

6.2.1 Vue RTL top

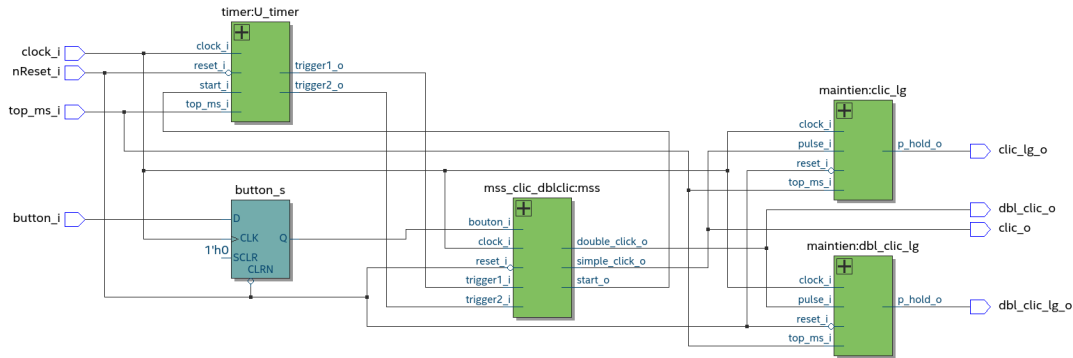


FIGURE 11 – Schéma RTL de l'entité top `det_clic_dbcl clic_top`

Le schéma obtenu correspond bien à celui que nous avons réalisé précédemment à la figure 7. Cela nous conforte dans l'idée que notre description VHDL suit correctement la solution prévue sur papier.

6.2.2 Vue RTL mss

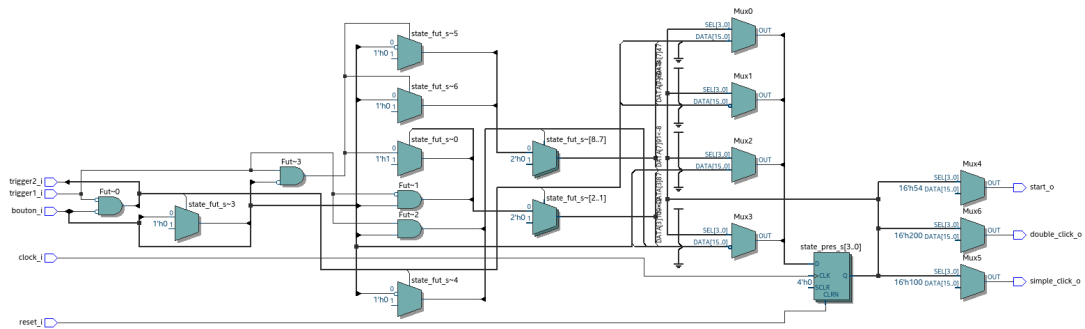


FIGURE 12 – Schéma RTL de la machine séquentielle `mss_clic_dbcl clic`

Bien que la MSS puisse sembler assez compliquée au premier abord, son fonctionnement reste relativement simple. Elle décide uniquement de l'état futur en fonction des entrées, puis met à jour son état présent et décode celui-ci afin de modifier les sorties. Une bonne nouvelle est que le schéma confirme bien qu'il s'agit d'une machine de Moore. Nous ne nous sommes donc pas trompés lors du développement de celle-ci.

La logique pourrait probablement être réduite avec un meilleur choix d'encodage des états, mais cet aspect n'a pas été pris en compte lors de l'étape d'implémentation.

6.2.3 Vue RTL timer

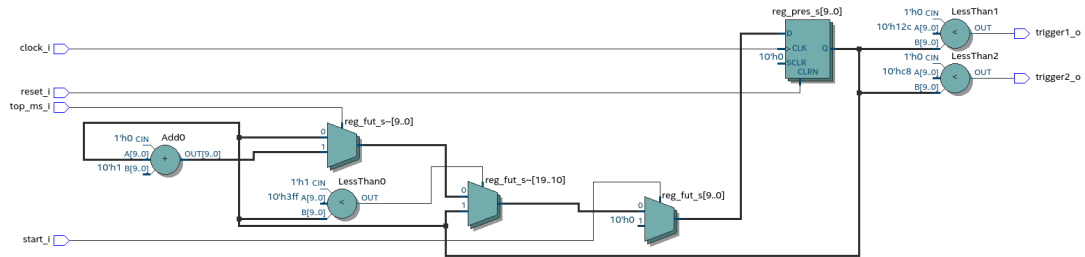


FIGURE 13 – Schéma RTL du timer

Le timer est lui aussi plutôt compréhensible. On remarque qu'il s'agit bien d'un compteur synchrone avec logique combinatoire de sortie. Les 3 MUX correspondent aux différents choix possibles dans le bloc `if-else` de notre code. Le bloc d'addition est nécessaire pour incrémenter un compteur sur 10 bits.

En revanche, la comparaison avec la valeur maximale peut sembler coûteuse. Il existe probablement une meilleure solution, car nous voulons simplement détecter le dépassement de la valeur maximale 1023. À ce moment-là, tous les bits sont à 1, donc une porte ET à 10 entrées devrait suffire et coûter moins cher qu'un comparateur complet.

6.3 Calcul de Fmax

Le synthétiseur est capable de calculer la Fmax dans le cas d'un système séquentiel entièrement synchrone pour une raison assez simple.

En effet, il connaît le temps de propagation des différents éléments logiques utilisés. En additionnant ces délais avec ceux des chemins combinatoires, qui peuvent être plus ou moins longs selon la logique générée, il peut déterminer le chemin critique du circuit. La période de la clock doit donc être plus grande que le temps nécessaire pour que ce chemin critique se stabilise. À partir de cette période minimale, le synthétiseur peut ensuite en déduire la fréquence maximale.

Il y a probablement aussi une marge de sécurité prise en compte dans ce calcul.

Au final, si on regarde de plus près la table 3, on remarque que plus il y a d'éléments logiques, plus la fréquence maximale risque d'être faible. Mais, cette relation est loin d'être linéaire et semble dépendre d'autres facteurs comme le placement et routage. On note que le timer seul et la MSS seule ont un nombre de LEs similaire mais une fréquence fortement différente. Cela pourrait être également dû au chemin critique comportant plus de LEs sur le timer que sur la MSS.

7 Simulation

Voici les logs de QuestaSim de la simulation de l'entité top.

```
# End time: 15:13:41 on May 27,2026, Elapsed time: 0:01:28
# Errors: 0, Warnings: 7
# vsim -voptargs="" +acc"" work.det_clic_dbclclcl_top_tb
# Start time: 15:13:41 on May 27,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.det_clic_dbclclcl_pkg
# Loading work.det_clic_dbclclcl_top_tb(struct)#1
# Loading work.det_clic_dbclclcl_top(struct)#1
# Loading work.timer(comport)#1
# Loading work.mss_clic_dbclclcl(state_machine)#1
# Loading work.maintien(comport)#1
# Loading work.det_clic_dbclclcl_top_tester(tester)#1
```

FIGURE 14 – Logs de compilation

```
VSIM 6> run -all
# ** Warning: NUMERIC_STD."=": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/UUT/dbl_clic_lg
# ** Warning: NUMERIC_STD."=": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/UUT/clic_lg
# ** Note: génération d'un clic
# Time: 10502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un double-clic
# Time: 510502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un clic
# Time: 1505502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un clic
# Time: 2014502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: Pas de détection de clic
# Time: 2525502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: Pas de détection de clic
# Time: 3027502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un double-clic
# Time: 3680502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: Pas de détection de double-clic
# Time: 4680502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un double-clic
# Time: 5682502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un clic
# Time: 6682502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# ** Note: génération d'un double-clic
# Time: 7183502 us Iteration: 0 Instance: /det_clic_dbclclcl_top_tb/tester
# 8184502000 ns >> fin de la simulation
# >> Vérifiez dans le log si il y des erreurs!
```

FIGURE 15 – Logs de simulation

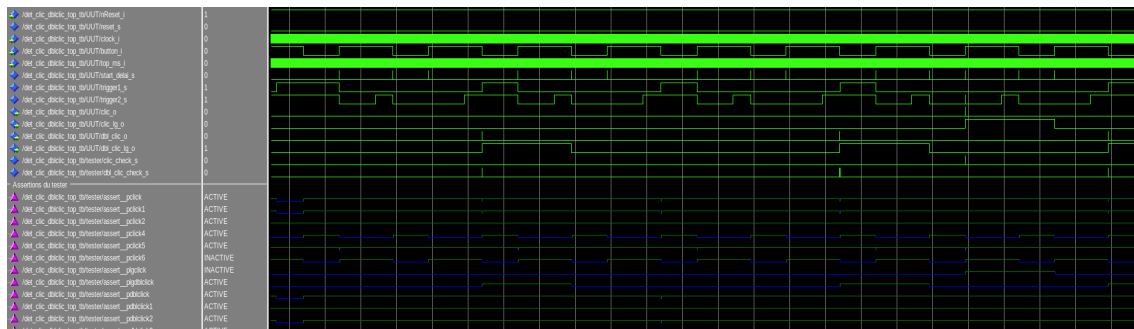


FIGURE 16 – Extrait du chronogramme de la simulation

Les avertissements affichés lors de la simulation sont dus au composant de maintien.

Comme on peut le voir dans les logs, aucune erreur n'a été levée ; on peut donc considérer que notre code a le comportement souhaité.

On peut également observer sur le chronogramme que la sortie *double-clic* est activée lors du relâchement du bouton, tandis que la sortie *clic simple* est activée lorsque le signal *trigger 2* est levé.

Lorsqu'une erreur survient, celle-ci apparaît sous la forme d'un triangle orange sur le chronogramme. Sur cet extrait, tout comme sur le reste du chronogramme, on constate qu'il n'y a plus d'erreurs.

8 Intégration carte

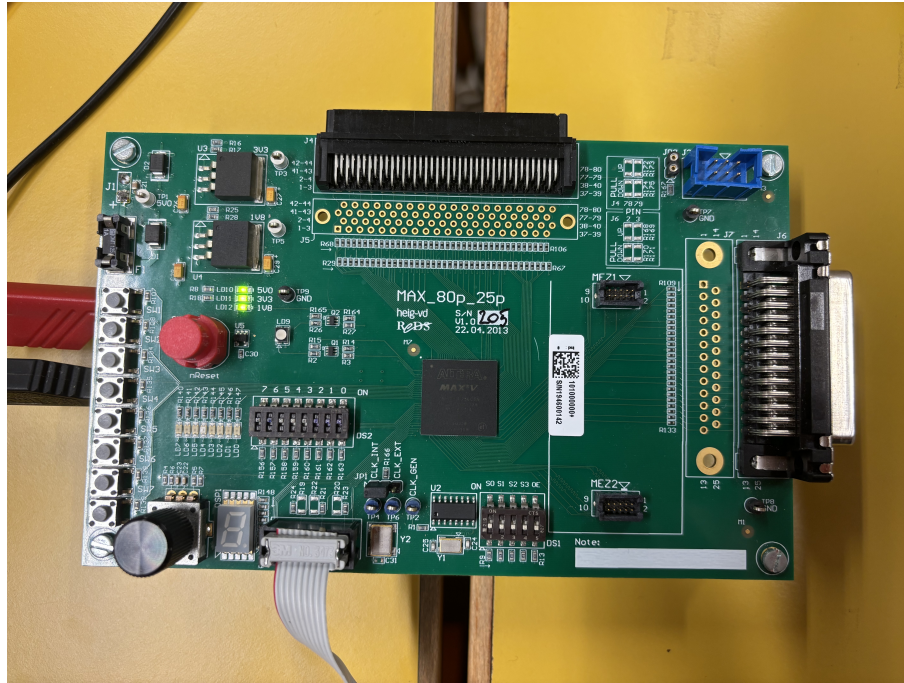


FIGURE 17 – MAX_80p_25p

- SW1 = bouton d'entrée
- LED0 = clic simple
- LED1 = double-clic
- LED2 = clic simple long
- LED3 = double-clic long

N°	Action sur SW1	LED2	LED3	Résultat attendu
1	Aucun appui sur SW1	Éteinte	Éteinte	Aucun clic détecté
2	Appui court puis relâchement	Allumée brièvement	Éteinte	Clic simple détecté
3	Deux appuis courts rapprochés	Éteinte	Allumée brièvement	Double-clic détecté
4	Appui maintenu trop longtemps	Éteinte	Éteinte	Aucun clic détecté
5	Premier appui court, puis deuxième appui court trop tardif	Allumée brièvement	Éteinte	Deux clics simples détectés
6	Premier appui court, puis deuxième appui maintenu trop longtemps	Éteinte	Éteinte	Aucun clic détecté
7	Plusieurs double-clics successifs	Éteinte	Allumée brièvement plusieurs fois	Détection répétée correcte

TABLE 4 – Tableau de test de la détection de clic et double-clic sur carte

Les signaux des LEDs 0 et 1 étant bien trop courts, ils ne sont pas vraiment utiles lors de cette phase d'observation, mais ils existent bien et sont repris sous forme maintenue sur les LEDs 2 et 3.

Il n'y a pas eu de problème lors de la programmation et tous les tests sont passés sans souci. À notre connaissance, aucun bug n'a été détecté sur la carte lors de la phase de test.

L'intégration a également été validée par Bruno Sutterlet le 26 mai 2026.

9 Conclusion

Dans ce laboratoire, nous avons réalisé un détecteur de clic simple et de double-clic composé d'une unité de traitement et d'une unité de contrôle.

Dans un premier temps, nous avons analysé le comportement attendu du système face aux différentes séquences de clic à l'aide de chronogrammes.

Nous avons ensuite identifié les blocs fonctionnels ainsi que leurs interactions, puis défini leur fonctionnement avant de les implémenter en VHDL. L'entité top-level a finalement été développée afin d'intégrer l'ensemble du système.

Les simulations automatiques nous ont permis d'identifier et de corriger plusieurs erreurs jusqu'à obtenir une exécution sans erreur provenant de notre code. Les avertissements restants sont liés au composant de maintien et n'empêchent pas la validation du comportement attendu.

Enfin, nous avons validé le projet en testant l'implémentation sur une carte matérielle, même si les cas limites étaient difficiles à observer précisément, car les entrées étaient générées manuellement. Nous avons aussi pu voir l'utilité des blocs de maintien, car les signaux de sortie n'auraient pas été visibles sans eux.

10 Signatures

Yverdon-les-Bains le 27 mai 2026

Fabien Léger & Jonathan Nicolet